

Binomial Tree Option Pricing Logic

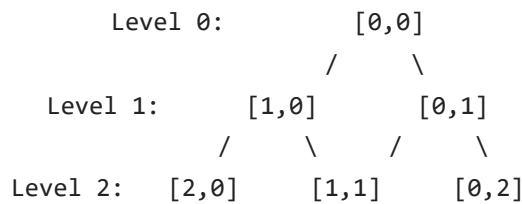
1. European Options: Tree Structure & Pricing

Node Representation

Consider a binomial tree where each node is represented as coordinates (i, j) :

- i = level or number of **up** moves
- j = level or number of **down** moves

Each node (i, j) can be stored as an element in a 2D matrix of size $(n + 1) \times (n + 1)$.



Probability and Payoff

- **Probability of each node path (P):** $\pi_u^i \cdot \pi_d^j$ (where π_u is the up probability and π_d is the down probability).
- **Payoff at maturity:** Based on call/put and strike price K :
 - Call Payoff: $\max(S_T - K, 0)$
 - Put Payoff: $\max(K - S_T, 0)$

Final Price Calculation

For calculating the price of the European option, we take the weighted sum of the final level (n), where each element satisfies $i + j = n$:

$$\text{Final Price of Option} = e^{-rT} \sum_{i+j=n} (\text{Probability of Node} \times \text{Payoff})$$

(Discounted back using the risk-free interest rate)

2. American Options: Node Structure & Pricing Logic

Node Configuration

Each node (i, j) in an American option tree contains:

- Up probability (π_u) and Down probability (π_d).
- **Boundary / Edge Rules:**
 - If node is for $i = 0$: Probability = π_d
 - If node is for $j = 0$: Probability = π_u
 - Otherwise (interior nodes): Store both probabilities (π_u and π_d).

Backward Induction & Implementation Logic

```
if (temp > tree[x][y].payoff)
{
    if (y == 0)
    {
        tree[x][y].final_payoff_node = temp * up_prob;
    }
    else if (x == 0)
    {
        tree[x][j].final_payoff_node = temp * down_prob;
    }
    else
    {
        if (tree[x-1])
        {
            // Evaluate previous tree level continuation
        }
    }
}
else
{
    tree[x][y].final_payoff_node = tree[x][y].payoff;
}
```